

ENCAPSULATION OFFLOADS

LCO, GSO_PARTIAL, TSO_MANGLEID, AND WHY LESS IS MORE

Alexander Duyck
Open Source Technologist
Intel Corporation
Alexander.Duyck@gmail.com

Agenda

- Understanding Checksums
- Understanding UDP Encapsulation
- Remote Checksum Offload (RCO)
- Local Checksum Offload (LCO)
- Partial Generic Segmentation Offload (GSO_PARTIAL)
- TCP Segmentation with IPv4 ID Mangling (TSO_MANGLEID)
- Why Less Is More
- Conclusions and Future Work

Understanding Checksums

- TCP, UDP, and IPv4 all use a one's complement sum
- 0 is equivalent to all 1's (negative zero), 0xFFFF == 0x0
- Byte order independent as carries from upper roll into lower
- Allows for any ordering as long as even/odd byte ordering respected
- Implemented on most architectures as an add with carry

Understanding Checksums

- Generating a one's complement checksum
  1. Start with checksum value of 0
  2. Perform one's complement sum of region
  3. Store the one's complement value of one's complement sum in checksum field

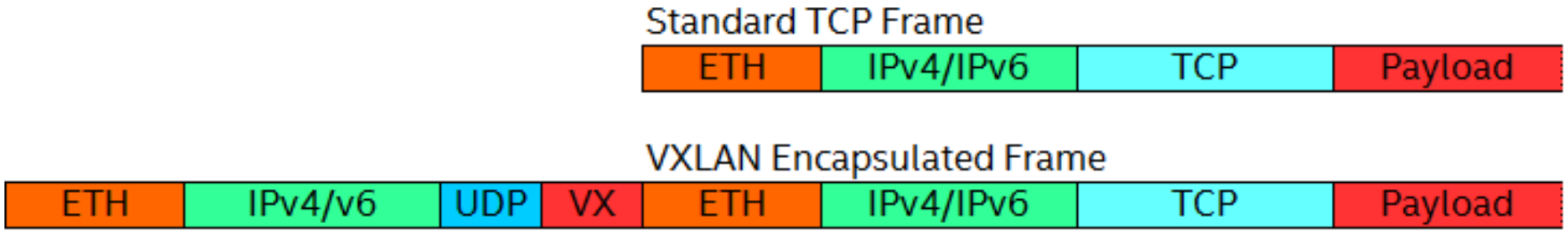
     If resultant checksum value is 0 replace it with all 1's value also known as "negative 0"
- Validating a one's complement checksum
  1. Perform one's complement sum of region
  2. If sum of region is all 1's (0xFFFF) then checksum is valid

Understanding Checksums

- TCP and UDP make use of pseudo-header as part of checksum
- Extra data included in sum that is not part of the transport data
- Source IPv4/v6 address
- Destination IPv4/v6 address
- IP Protocol number as 0 padded 16 bit value
  - Same byte ordering as other fields
- Segment length, length from start of transport to end of packet
- Sum of header usually stored in L4 checksum value instead of 0

Understanding UDP Encapsulation

- Take a fully formed Ethernet Frame and place it inside of a UDP datagram

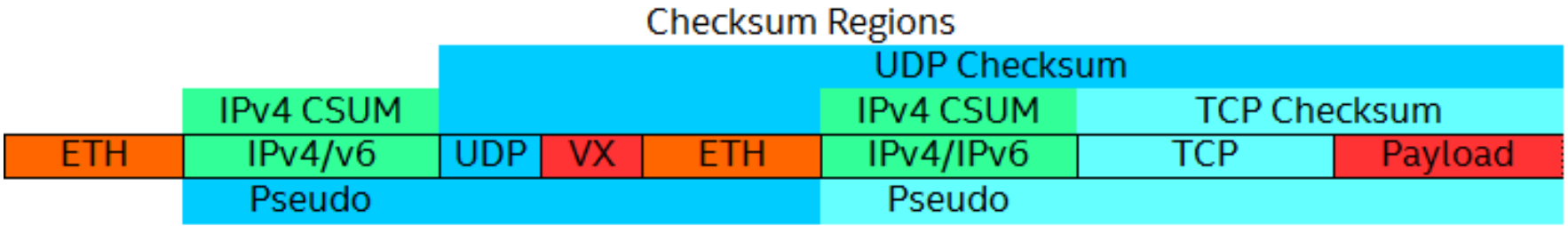


Understanding UDP Encapsulation

- Cannot parse Inner headers without knowing packet is a tunnel
  - No parsing means no checksum offload on inner headers
  - Cannot segment frames if hardware cannot recognize frame type
- Devices only support one checksum on Tx
  - Can only offload one checksum, default to outer
  - Cannot perform TSO if outer checksum is enabled

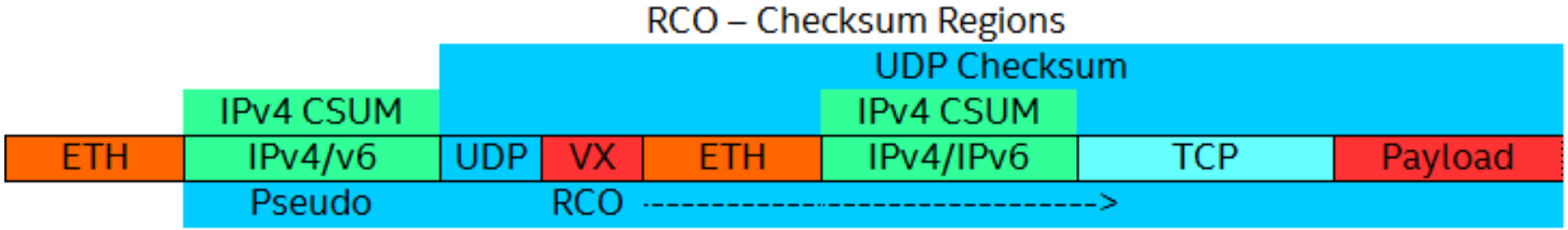
Understanding UDP Encapsulation

- In general if outer checksum is enabled it overlaps with inner checksums



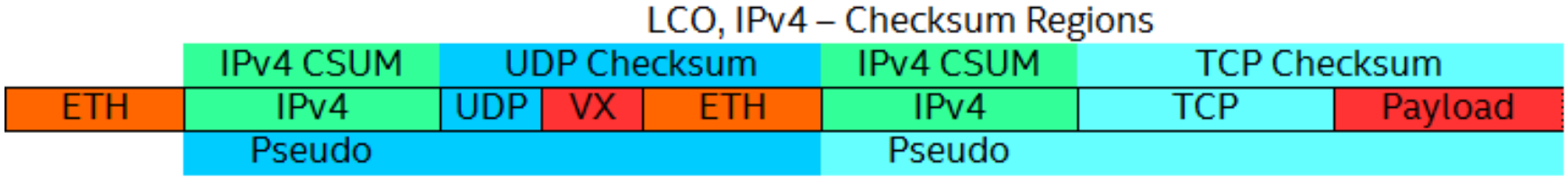
Remote Checksum Offload (RCO)

- Instead of performing 2 checksums only perform the outermost checksum
- Include metadata with tunnel header to identify start and offset of checksum
- Unpack checksum on Rx and place in correct spot in inner header



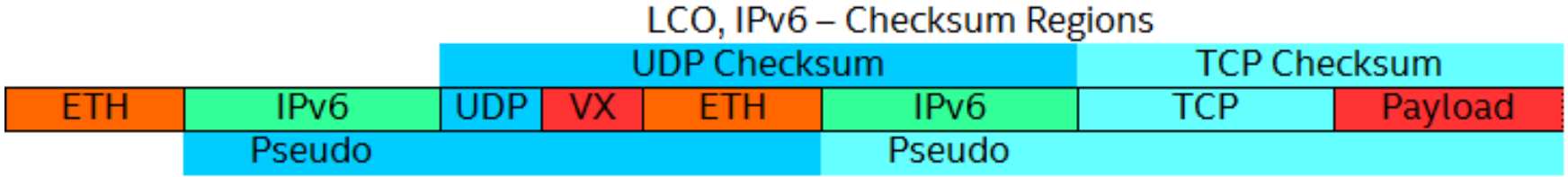
Local Checksum Offload (LCO)

- We know what the checksum for IP headers and L4 headers should be
  - IPv4 sum should be 0xFFFF if checksum is correct
  - UDP/TCP checksum should be ~(pseudo header sum)
- If we assume those values we can perform software checksum smaller area
  - Allows us to focus hardware checksum on inner payload data



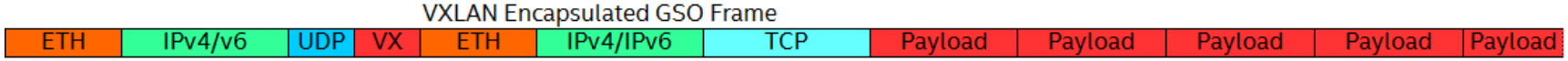
Local Checksum Offload (LCO)

- We know what the checksum for IP headers and L4 headers should be
  - IPv4 sum should be 0xFFFF if checksum is correct
  - UDP/TCP checksum should be ~(pseudo header sum)
- If we assume those values we can perform software checksum smaller area
  - Allows us to focus hardware checksum on inner payload data



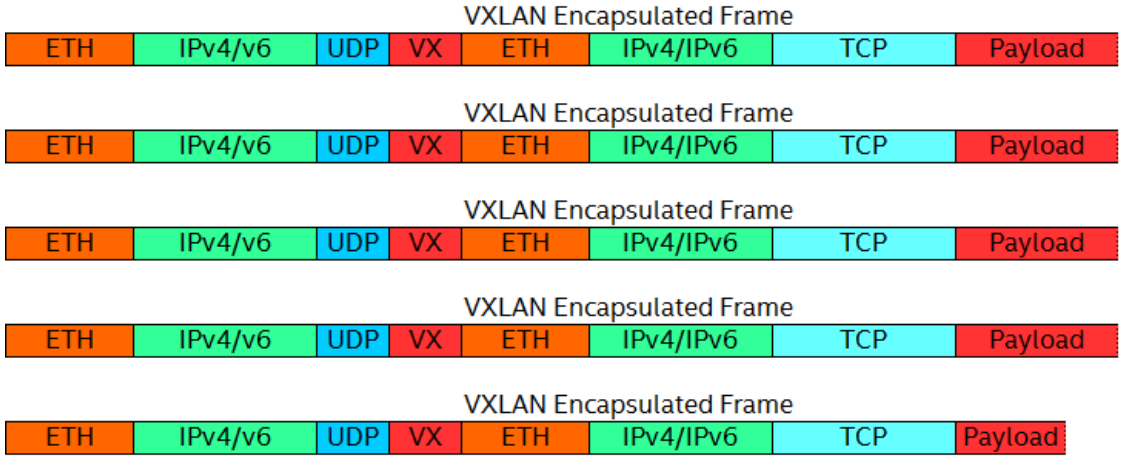
Generic Segmentation Offload (GSO) / TCP Segmentation Offload (TSO)

- TSO supported on most devices that use 0 for outer checksum
- GSO can now at least make use of LCO to reduce overhead



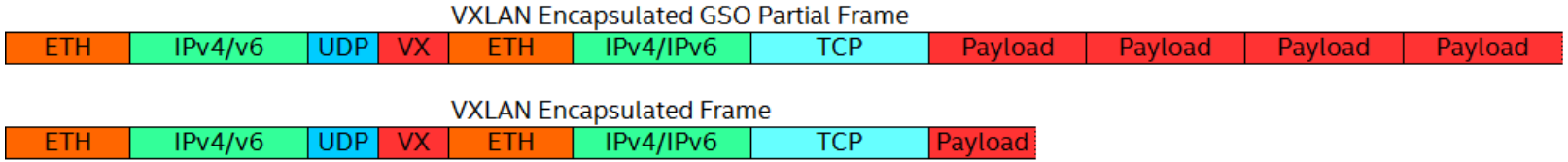
Generic Segmentation Offload (GSO) / TCP Segmentation Offload (TSO)

- All frames with the same size have the same UDP checksum
  - IPv4 checksum cancelled out IPv4 ID
  - TCP checksum cancelled out TCP header changes



Partial Generic Segmentation Offload (GSO_PARTIAL)

- Hardware just replicates value stored in UDP checksum
- All headers before csum_start are representative of a single frame
- Possibility of replicating other headers

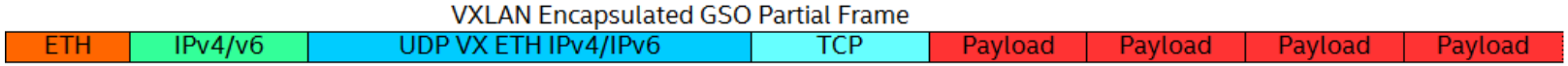


TCP Segmentation with IPv4 ID Mangling (TSO_MANGLEID)

- IPv6 contains no fields that vary between segmented frames
- IPv4 only has the ID field that varies between frames
- TCP flows require DF bit to be set so we should never fragment inner headers at the inner network level.
- RFC 6864 States
  - All devices that examine IPv4 headers MUST ignore the IPv4 ID field of atomic datagrams.
- Provided a means of enabling this via NETIF_F_TSO_MANGLEID flag
  - Not enabled by default due to concerns about GRO/GSO altering frames

TCP Segmentation with IPv4 ID Mangling (TSO_MANGLEID)

- The drivers igb, igbvf, ixgbe, ixgbevf all had support for “IP-in-IP tunneling”
  - Variable IP options length up to 511 bytes in size for IP header
  - Support for outer header as long as no inner network header modified
  - IPv4 checksum needed to be seeded with checksum for “options”



GRO with Support for Fixed IPv4 ID Field (SKB_GSO_TCP_FIXEDID)

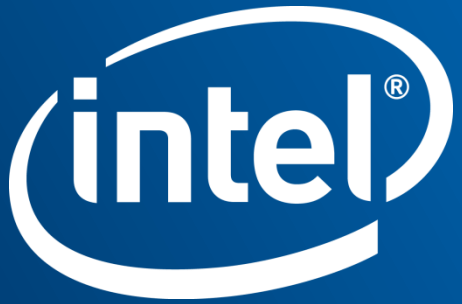
- GRO was expecting incrementing IPv4 ID field, as such it had to be updated
  - In order to support GRO we added SKB_GSO_TCP_FIXEDID
  - Those using IPv6 to IPv4 translation saw gains from this as well

Why Less is More

- Everyone SHOULD support NETIF_F_HW_CSUM
- CHECKSUM_UNNECESSARY is a pain to deal with
- Parsing based offloads are unreliable
  - Only ever provide support for some fixed number of ports
  - Parsers built into the hardware cannot adapt

Conclusions and Future Work

- Half-measures like GSO partial can provide significant gains
  - Using outer checksum allows for nearly doubling Rx throughput
  - Significant Tx throughput improvement thanks to GSO partial
- Related work being done by others
  - Support partial splitting at frag_list pointer – Steffen Klassert
- Enable TSO_MANGLEID by default?


intel®